

Ensuring Database Integrity: A Comprehensive Analysis of Watchdog Mechanisms for PostgreSQL High Availability and Split-Brain Prevention

High availability in distributed database systems represents a perpetual balancing act between maximizing operational uptime and strictly guaranteeing data consistency. In the context of PostgreSQL, achieving continuous availability requires a robust automated failover mechanism to seamlessly transition client traffic from a failed primary database to a synchronized replica. However, the introduction of automated failover logic simultaneously introduces the most critical vulnerability inherent in any distributed data architecture: the split-brain scenario. When an unpredictable network partition occurs, an isolated primary node and a newly promoted standby node may simultaneously believe they hold authoritative control over the cluster. If client routing logic is equally partitioned, both nodes will independently accept write transactions, leading to diverging transaction timelines, irreconcilable data corruption, and catastrophic business impact.

To structurally eliminate the possibility of split-brain occurrences, distributed consensus systems employ node fencing, specifically STONITH (Shoot The Other Node In The Head), to aggressively isolate or terminate the rogue primary node before the new primary is permitted to accept connections. Within modern Linux environments, the most reliable local enforcement mechanism for STONITH is the system watchdog. This exhaustive technical report examines the architecture of software and hardware watchdog solutions within the Linux kernel, common operational use cases, and their specific, highly technical integration into PostgreSQL high availability stacks. A deep architectural focus is placed on Patroni, alongside comparative analyses of Pacemaker with Corosync, pg_auto_failover, and repmgr.

Part 1: Watchdog Software and Hardware Solutions under Linux

A watchdog timer operates as an uncompromising fail-safe continuous monitoring system, fundamentally functioning as a deadman switch for the operating system and critical applications. It is a hardware or software mechanism that requires periodic servicing—often referred to as kicking, petting, or feeding—by the operating system kernel or a designated userspace daemon. If the watchdog is not serviced within a strictly defined temporal threshold, the watchdog assumes the system has experienced an unrecoverable hang, deadlock, or starvation event, and it forcefully triggers a complete system reset to restore the system to a known good state¹.

Hardware Watchdog Architecture and Efficacy

Hardware watchdogs represent the absolute gold standard for system reliability and fencing enforcement. They are typically implemented as physical components directly on the host motherboard, within embedded microcontrollers, or as supplementary add-on cards¹. Because they exist at the bare-metal hardware layer, their operation is entirely independent of the operating system's kernel state².

If a catastrophic failure occurs at the software layer—such as an unrecoverable kernel panic, a total CPU lockup induced by a faulty driver, or a severe out-of-memory starvation event where the operating system scheduler completely ceases to function—the hardware timer continues to count down relentlessly. Once the timer reaches zero, the hardware electrically asserts the reset pin of the CPU, initiating an immediate, ungraceful reboot that instantly severs all network and storage I/O².

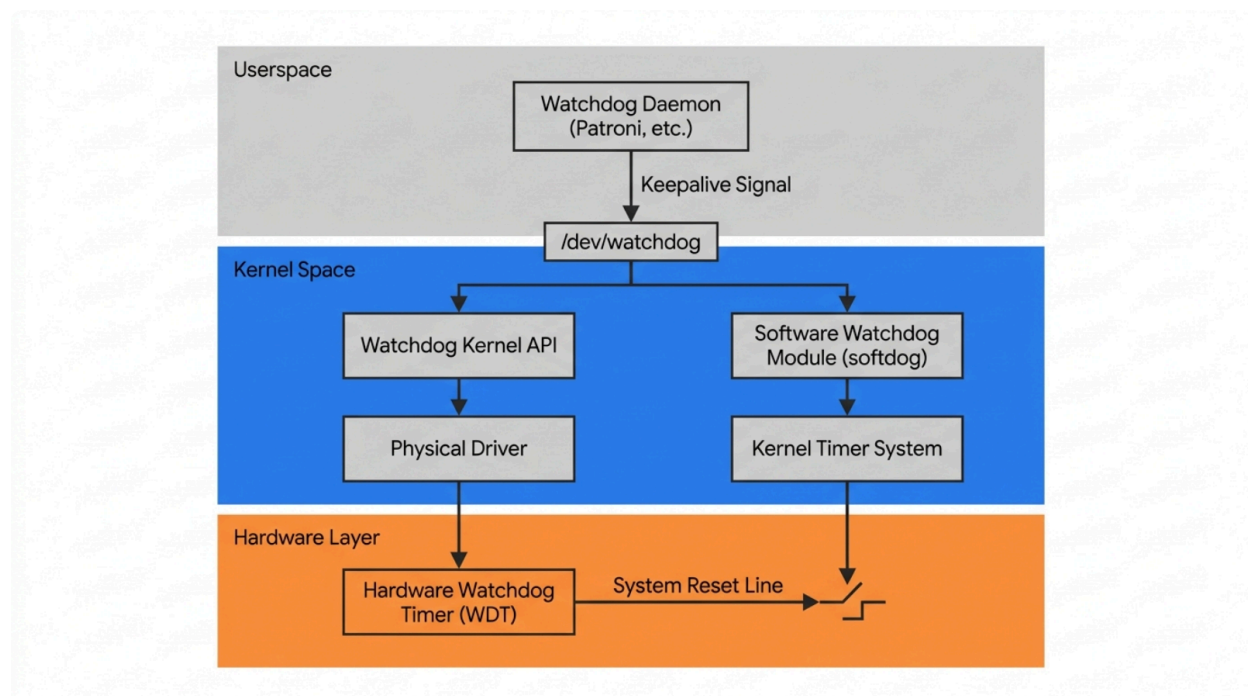
The Linux kernel supports a wide array of hardware watchdog drivers tailored to specific enterprise and embedded chipsets. Common implementations include the Intel TCO (Total Cost of Ownership) watchdog timer, designated by the `iTCO_wdt` module, which is ubiquitous on modern Intel server architectures. Similarly, AMD platforms utilize the `sp5100_tco` module for SP5100 and associated chipsets. Other notable hardware drivers include the `i6300esb` for older Intel platforms, and the `ipmi_watchdog`, which leverages the Baseboard Management Controller via the Intelligent Platform Management Interface, a staple in enterprise data center management². Additionally, Hewlett Packard Enterprise provides the `hpwdt` driver (the iLO NMI Watchdog), which possesses the advanced capability to generate a Non-Maskable Interrupt prior to executing the hard reset, potentially allowing the kernel to capture a crash dump for post-mortem analysis before the power cycle occurs⁹.

Software Watchdog Emulation

In increasingly prevalent virtualized environments, public cloud infrastructures, and containerized deployments, exposing a physical hardware watchdog device to the guest operating system is frequently impractical, unsupported, or explicitly blocked by the hypervisor. To bridge this critical architectural gap, the Linux kernel provides the `softdog` module, a software-based emulator that replicates hardware watchdog functionality utilizing the kernel's internal timer infrastructure².

When the `softdog` kernel module is loaded, it creates the standard `/dev/watchdog` character device interface, rendering it indistinguishable from a hardware device from the perspective of userspace applications. When a high-availability daemon opens this device and periodically writes a service pulse, the `softdog` module reschedules an internal kernel timer. If the userspace daemon hangs, crashes, or is starved of CPU cycles and fails to write to the device, the internal timer expires, deliberately triggering a kernel panic and a subsequent system reboot¹.

Execution Pathways of Hardware vs. Software Watchdogs



Hardware watchdogs operate independently at the physical layer, capable of resetting a frozen system regardless of kernel state. Conversely, the softdog module relies on internal kernel timers, leaving it vulnerable to total CPU hard lockups.

However, it is crucial for systems architects to understand that softdog operates strictly within the operational limitations of the Linux kernel itself. Red Hat and other enterprise vendors explicitly warn that the softdog driver fundamentally assumes that at least one CPU core remains responsive and capable of scheduling processes and handling interrupts. In the event of a severe hardware fault, an uninterruptible sleep deadlock, or a total hard lockup where all CPUs are completely unresponsive to internal timer interrupts, the code pathway within the softdog driver responsible for actually executing the system reboot will never be reached, rendering the fail-safe useless². Despite this structural limitation, softdog remains universally supported across all Linux distributions and serves as the standard fallback mechanism for virtual machines to catch subtle software failure modes, such as deadlocks, isolated infinite loops in userspace routing daemons, or driver-level CPU blocking⁴.

Kernel Lockup Detection Mechanisms

Operating alongside the explicit watchdog infrastructure, the Linux kernel includes supplementary, built-in diagnostic mechanisms designed to detect various system freeze states and optionally trigger recovery actions. These detectors operate at an even lower level than userspace watchdogs.

The Soft Lockup Detector monitors whether a CPU core is executing kernel code for an

extended duration without voluntarily yielding to the process scheduler. The detector functions by running a high-priority kernel thread on each available CPU core. If this specific thread does not get scheduled for a configurable period—which defaults to twenty seconds, calculated as $2 * \text{watchdog_thresh}$ —the kernel logs a severe warning. Depending on the `kernel.softlockup_panic` sysctl configuration, the system can be configured to dump all CPU backtraces and immediately trigger a kernel panic to force a reboot¹⁰.

For more severe freezes, the Hard Lockup Detector utilizes hardware performance counters to trigger Non-Maskable Interrupts. If a CPU core does not respond to standard hardware interrupts within the defined `watchdog_thresh` limit (typically ten seconds), a hard lockup is declared. Because this utilizes NMIs, it can often interrupt code that has maliciously or accidentally disabled standard interrupts¹⁰. Finally, the Hung Task Detector identifies individual processes that have remained stuck in an uninterruptible sleep state for longer than expected, typically defaulting to 120 seconds. While a hung task does not mean the CPU is locked, it usually indicates a catastrophic failure in an underlying storage device, a network filesystem, or a kernel block driver, which may eventually cascade into a full system failure¹⁰.

The Linux Watchdog Application Programming Interface

Interacting with the watchdog in a Linux environment occurs exclusively via the `/dev/watchdog` (or occasionally `/dev/watchdog0`) character device file. The interaction between a userspace daemon and the kernel driver is governed by a specific Application Programming Interface heavily reliant on `ioctl` (input/output control) system calls, which allows for complex signaling beyond simple byte writes¹⁴.

The watchdog is immediately armed the moment the `/dev/watchdog` file descriptor is successfully opened by a userspace application. Once armed, the countdown begins¹⁴. To service the timer, the application can either write raw data to the device or, more formally and reliably, send the `WDIOC_KEEPALIVE` `ioctl` command. This command serves as the explicit heartbeat, instructing the driver to reset the countdown timer back to its maximum threshold¹⁴. A critical component of this API is the "Magic Close" feature. By default, simply closing the file descriptor will not disable the watchdog; the kernel assumes the userspace application crashed unexpectedly, and the system will proceed to reboot once the timer expires. To gracefully disarm the watchdog—for instance, during a planned service shutdown or server maintenance—the application must explicitly write the ASCII character `V` into the file descriptor immediately before closing it¹⁴. However, administrators can override this behavior by compiling the kernel with the `CONFIG_WATCHDOG_NOWAYOUT` option. If this option is active, the watchdog cannot be disabled under any circumstances once it has been armed, bypassing even the Magic Close feature. This strict enforcement ensures that an accidentally killed daemon or a maliciously terminated high-availability agent definitively results in a system reboot, trading operational convenience for absolute, uncompromising data safety⁹.

The API also allows the daemon to query and modify the operational parameters of the hardware dynamically. The timeout interval can be retrieved using `WDIOC_GETTIMEOUT` and modified using `WDIOC_SETTIMEOUT`, allowing the userspace high-availability daemon to dictate the hardware's tolerance based on application-specific timing logic and network

latency profiles¹⁵. Furthermore, the daemon can query the specific capabilities of the underlying driver using `WDIOC_GETSUPPORT`, which returns a `watchdog_info` data structure detailing the driver's options, firmware version, and identity string⁹.

Global Watchdog Management via systemd

In contemporary Linux distributions, the `systemd` init system possesses deep, native integration with hardware watchdog timers, acting as the ultimate supervisor for the entire operating system stack. By modifying the global `/etc/systemd/system.conf` configuration file, system administrators can instruct `systemd` itself (running as PID 1) to act as the primary watchdog feeder.

The `RuntimeWatchdogSec=` directive specifies the timeout interval for the hardware watchdog. If configured to a value such as thirty seconds, `systemd` will open `/dev/watchdog0` and automatically feed the device at half the configured interval (every fifteen seconds). If the Linux kernel hangs entirely, or if `systemd` itself deadlocks and fails to send the `WDIOC_KEEPALIVE` ioctl, the hardware will physically reset the system¹⁹.

Furthermore, `systemd` provides critical fail-safes during the system teardown process via the `RebootWatchdogSec=` and `ShutdownWatchdogSec=` directives. These parameters control the watchdog interval during a reboot or power-off sequence. If a software-initiated reboot hangs indefinitely—a common occurrence when dealing with frozen network-attached storage mounts or unresponsive hypervisor drivers—the watchdog hardware will force a hard reset after this duration (which defaults to ten minutes), ensuring the machine does not remain stranded in an unusable, partially-shutdown state²⁰.

Systemd Directive	Scope of Supervision	Description	Default Action upon Failure
<code>RuntimeWatchdogSec=</code>	Global (PID 1)	Configures the hardware watchdog timeout. Systemd feeds the device periodically.	Hardware-enforced system reset.
<code>RebootWatchdogSec=</code>	Global (Shutdown)	Ensures a system reboot completes within a specified timeframe.	Hardware-enforced system reset.
<code>WatchdogSec=</code>	Service Level	Expects the managed service to send <code>sd_notify("WATCHDOG=1")</code> .	Systemd terminates service with <code>SIGABRT</code> and applies restart policy.

Beyond managing the global hardware timer, systemd extends software-level watchdog capabilities to individual managed services through the `WatchdogSec=` directive placed inside a specific unit file. When this is configured, systemd provides the environment variable `WATCHDOG_USEC` to the application, dictating the expected heartbeat frequency. The application must then periodically invoke the `sd_notify("WATCHDOG=1")` API call. If the service experiences a deadlock, infinite loop, or resource starvation and fails to check in with systemd in time, the init system forcefully terminates the offending process with a `SIGABRT` signal. Following termination, systemd will apply the defined `Restart=` policy, thereby preventing "zombie" processes from lingering silently in the background and accumulating degraded states²².

Part 2: The Split-Brain Threat in PostgreSQL

PostgreSQL is a fundamentally robust relational database, but it does not natively include an automated failover clustering manager within its core binary. While its native streaming replication effectively guarantees data synchronization from a primary to one or more standbys—either asynchronously for performance or synchronously for absolute durability—external orchestration tooling is strictly required to detect a primary failure and seamlessly promote a standby to take over write operations³.

The introduction of external automation creates the primary risk vector for distributed data systems: the split-brain scenario. Consider a deployment where a network partition severely degrades the infrastructure, effectively isolating the primary database node from the rest of the cluster and the orchestration manager, but leaving the primary node fully operational and connected to a subset of application servers. If the orchestrator detects the loss of communication with the primary, it will promote a standby to assume the primary role³. Because the original primary is unaware of its demotion due to the network partition, it will continue to accept and commit write transactions from the applications it can still reach. Simultaneously, the newly promoted primary will accept transactions from the applications routed to it. The transaction timelines of the two databases will immediately and irreversibly diverge. Once the network partition heals, it is technologically impossible for PostgreSQL to simply merge the diverging transaction logs, resulting in permanent data inconsistency, logical corruption, and massive business liability³. Therefore, automated failover is fundamentally unsafe unless it is strictly coupled with a node fencing mechanism that absolutely guarantees the old primary is completely neutralized before any new primary is permitted to accept network traffic³⁰.

Part 3: Patroni's Architecture and Watchdog Integration

In the modern ecosystem, Patroni has emerged as the definitive industry standard for PostgreSQL high availability. Originally developed by Compose as a fork of the Governor project and currently maintained by a robust open-source community heavily backed by

Zalando, Patroni operates as a Python-based daemon that runs alongside and directly supervises each PostgreSQL instance in the cluster³.

Unlike older legacy systems that rely on peer-to-peer voting, Patroni delegates cluster state management to a highly available Distributed Configuration Store, supporting etcd, HashiCorp Consul, Apache ZooKeeper, or the Kubernetes API²⁵. These DCS solutions leverage distributed consensus algorithms, such as Raft or Paxos, to ensure that the global state of the database cluster is mathematically consistent across all nodes, preventing local network blips from causing differing views of reality³.

The Patroni Split-Brain Protection Paradigm

Patroni's primary architectural defense against split-brain scenarios relies on a distributed, temporary lease mechanism maintained within the DCS. For a PostgreSQL node to operate as the primary, its local Patroni agent must continually acquire and successfully renew a "leader key" in the DCS³. This key has a strict Time-To-Live expiration. If the primary Patroni agent fails to renew this key before the TTL expires, the DCS automatically releases the lock. At this exact moment, the remaining Patroni agents on the standby nodes detect the missing lock and initiate a race to acquire it, evaluating replication lag to ensure the most advanced standby is promoted to the new leader⁵.

Under normal, graceful operation, if the Patroni daemon on the primary node realizes it cannot reach the DCS to renew its lease—perhaps due to a severed network uplink—it will proactively and gracefully demote its managed PostgreSQL instance, restarting it as a read-only replica, or shutting it down entirely, before the DCS TTL expires²⁶.

However, relying solely on this software-level graceful demotion is perilous, as it is vulnerable to a multitude of severe system-level anomalies where graceful execution is impossible:

1. **Patroni Daemon Crash:** The Python-based Patroni process itself may crash due to an unhandled exception, a memory leak, or be abruptly terminated by the Linux Out-Of-Memory Killer during a period of high database load²⁶.
2. **PostgreSQL Unresponsiveness:** The command to shut down PostgreSQL via SIGTERM or SIGKILL might take an inordinate amount of time to execute due to heavy disk I/O flushing, or the core postmaster process might become entirely unkillable due to being stuck in an uninterruptible sleep state waiting on a faulty storage driver²⁶.
3. **Hypervisor Interference:** In virtualized environments, a virtual machine hosting the primary node might be temporarily paused for live migration across physical hosts, or its execution frozen due to hypervisor CPU scheduling contention. When the virtual machine is eventually unpaused, the local Patroni agent and the PostgreSQL instance may act as if no time has passed, believing they still hold the leader lock. In reality, the DCS has long since expired the lease in real-time and promoted a different node, resulting in immediate split-brain upon network resumption⁶.

Temporal Mathematics of the Watchdog Fail-Safe

To categorically guarantee that a demoted or isolated primary cannot process transactions under any physical or virtual circumstance, Patroni directly integrates with the Linux

/dev/watchdog character device³.

Patroni aggressively arms the watchdog prior to ever attempting to promote a PostgreSQL instance to a primary role. If the administrator has configured `watchdog: mode: required` in the `patroni.yml` configuration file, and the `/dev/watchdog` device cannot be successfully opened or initialized, Patroni will outright refuse to participate in leader elections. This design principle strictly prioritizes data consistency over database availability²⁶.

The efficacy of this integration relies on stringent temporal mathematics tied to the Patroni high availability control loop. Four primary variables dictate this safety envelope:

- `ttl` (default 30 seconds): The duration the leader lock persists in the DCS without requiring renewal from the primary²⁶.
- `loop_wait` (default 10 seconds): The sleep interval between Patroni's HA iterations where it checks health and attempts DCS renewals²⁶.
- `retry_timeout` (default 10 seconds): The timeout threshold for attempting network operations against the DCS²⁶.
- `safety_margin` (default 5 seconds): The critical buffer period reserved between updating the DCS leader key and servicing the watchdog keepalive²⁶.

The chronological sequence of a failure event demonstrates how these variables prevent split-brain. Assuming default values, consider a scenario where a total network partition occurs at $T=0$ seconds immediately after Patroni successfully renewed its lock. At $T=10$ seconds, the `loop_wait` expires, and Patroni attempts its next HA loop but fails to reach the DCS due to the partition. Patroni will attempt to gracefully demote PostgreSQL. However, if the system is completely frozen, Patroni cannot execute the demotion. By default, Patroni has configured the hardware watchdog timeout to expire exactly `safety_margin` (5 seconds) before the DCS `ttl` expires. Therefore, at $T=25$ seconds, the unserviced hardware watchdog expires, and the motherboard physically forces a system reboot, instantly killing the rogue PostgreSQL instance. Finally, at $T=30$ seconds, the DCS `ttl` expires, the lock is released, and the surviving standbys are permitted to initiate a new leader election.

Because the watchdog forced a hard reset at $T=25$ seconds, the architecture guarantees that the rogue primary node is hard-fenced before the standbys are permitted to promote, preventing any overlapping write windows²⁶.

To achieve mathematically absolute guarantees—especially against severe hypervisor pauses that might suspend the guest operating system at the exact millisecond between a DCS update confirmation and the subsequent watchdog `ioctl` tick—the `safety_margin` configuration can be explicitly set to `-1`. Patroni interprets this special value by automatically setting the hardware watchdog timeout to exactly half of the TTL (`ttl // 2`). This aggressive posture ensures that even extreme edge-case timing anomalies result in a hard reset well before a split-brain condition can actualize²⁶.

Technical Implementation Details in the Patroni Codebase

At the codebase level, Patroni interacts with the Linux watchdog API through its `patroni.watchdog.linux` Python module. Because standard Python libraries do not natively map to low-level Linux `ioctl` structures, Patroni must manually construct the `ioctl` byte commands

utilizing the ctypes and fcntl libraries, wrapping calls to WDIOC_KEEPLIVE and WDIOC_SETTIMEOUT³⁷.

The complexity of this low-level implementation is evident when analyzing cross-platform compatibility. The standard kernel macro layouts for IOC_NONE, IOC_WRITE, and IOC_READ define byte shifting operations that determine how data is passed between userspace and the kernel. On most standard x86 and ARM architectures, IOC_READ maps to the value 2, and IOC_WRITE maps to 1. However, on specific PowerPC architectures (ppc64, ppc64le), these constants are historically inverted (IOC_WRITE equals 4, IOC_READ equals 2)¹⁸.

A highly specific historical bug (#2265) within the Patroni codebase caused watchdog initialization to fail catastrophically on Red Hat Enterprise Linux 8 ppc64le systems. When Patroni attempted to invoke the WDIOC_GETSUPPORT command to query the driver's capabilities, it utilized the incorrect bitmask, resulting in the kernel rejecting the call with [Errno 25] Inappropriate ioctl for device¹⁸. To resolve this, the Patroni codebase was updated to explicitly invoke platform.machine(), checking if the architecture is mips, sparc, powerpc, ppc64, or ppc64le, and applying custom bit offsets to ensure stable and correct ioctl communication across all enterprise hardware³⁸.

Further technical nuances involve Patroni's internal threading model. In Python versions 3.11 and newer, running Patroni on memory-restricted systems with aggressive memory overcommit settings (e.g., vm.overcommit_memory=2, which is ironically recommended for PostgreSQL to prevent the OOM killer) resulted in unpredictable MemoryError exceptions when Patroni attempted to spawn new threads dynamically. This caused the Patroni REST API to become completely unresponsive, even while the PostgreSQL database itself remained healthy, leading load balancers to falsely declare the primary dead³¹. To address this stability issue, newer versions of Patroni completely refactored the codebase to avoid starting and stopping threads at runtime, introducing a persistent thread pool architecture for the REST API and asynchronous tasks, configurable via restapi.thread_pool_size³⁵.

When Patroni determines it is safe to gracefully demote the database, or when a database administrator initiates a manual failover or pauses the cluster for maintenance, Patroni invokes the close() method on its internal LinuxWatchdogDevice object. This method deliberately sends the "Magic Close" character to disarm the hardware timer. This action explicitly prevents the watchdog from firing, allowing the physical server node to remain online and operational without hosting the primary database²⁶.

Operational Configuration for Patroni Watchdogs

For Patroni to successfully manage the watchdog, standard Linux file permissions must be meticulously addressed. By default, the /dev/watchdog character device is owned by the root user. Because Patroni operates securely under the unprivileged postgres system user, administrators must modify ownership using chown postgres /dev/watchdog or, preferably, apply persistent udev rules to ensure Patroni can successfully execute open(), ioctl(), and write() operations on the device file across reboots²⁶.

Furthermore, network glitches can cause unnecessary failovers if the timeout values are too aggressive. When utilizing Consul as the DCS, Patroni administrators frequently utilize the

consul.checks:[] configuration to disable native Consul serf checks, relying entirely on Patroni's internal ttl, retry_timeout, and loop_wait metrics to determine node viability, thereby increasing resilience against micro-outages. Administrators also tune the Consul server raft_multiplier to adjust the frequency of messaging between cluster members, ensuring that minor network latency does not trigger an unwarranted database failover⁴².

When deploying the softdog kernel module for testing or within virtualized environments, the module must be explicitly loaded using modprobe softdog. To facilitate safe testing without inducing actual server reboots, administrators can pass the soft_noboot=1 parameter to the modprobe command. In this diagnostic mode, when the watchdog expires, it bypasses the system reset and simply logs a warning line in the kernel ring buffer, visible via the dmesg command, allowing engineers to validate their timeout math without causing disruptions²⁶.

Part 4: Alternative PostgreSQL HA Solutions and Fencing

While Patroni relies on a dedicated Distributed Configuration Store to manage consensus, other enterprise architectures take distinctly different approaches to PostgreSQL high availability and split-brain mitigation.

Pacemaker, Corosync, and SBD Integration

In traditional, highly regulated enterprise environments, systems administrators frequently deploy the proven Pacemaker and Corosync stack for global resource management and high-availability clustering. In this mature architecture, PostgreSQL is not managed by a dedicated Python daemon but rather via OCF (Open Cluster Framework) resource agents, such as pgsqlms or PostgreSQL Automatic Failover (PAF)⁴⁴.

Unlike Patroni's lease-based demotion, Pacemaker strictly and unapologetically enforces STONITH using dedicated cluster fencer daemons. If network communication is lost between data centers, Corosync calculates quorum based on expected votes. The partition possessing the majority quorum will actively instruct the fencing daemon to terminate the nodes stranded in the minority partition²⁷.

Storage-Based Death (SBD) is the premier fencing mechanism utilized in these environments to deeply integrate watchdog timers with Pacemaker²⁷. SBD is designed to operate in two distinct modes depending on infrastructure availability:

1. **Disk-Based SBD:** In environments with Fibre Channel or iSCSI, nodes exchange targeted "poison pill" messages via a highly available shared block storage device. The SBD daemon runs on each node, continuously monitoring these disk slots while simultaneously feeding the local hardware watchdog. If a node reads a "poison pill" specifically targeted at its own ID by the cluster manager, the SBD daemon immediately ceases feeding the watchdog, intentionally allowing the hardware to reboot the machine to satisfy the STONITH request⁷.
2. **Diskless SBD:** In modern shared-nothing architectures lacking a centralized SAN, SBD relies entirely on the watchdog timer and network quorum calculations. Because a simple two-node cluster cannot establish quorum during a network split, Diskless SBD requires

a highly resilient network tie-breaker, typically implemented as a Corosync QDevice operating on a third, lightweight witness node. If a database node loses quorum, or if Pacemaker determines the node must be fenced due to an unrecoverable daemon failure, the local SBD daemon self-terminates or stops servicing the timer, guaranteeing a system reset without relying on external power distribution units⁸.

To integrate SBD with Pacemaker seamlessly, administrators must explicitly configure the cluster property `stonith-watchdog-timeout`. Mathematical precision is vital here: this value is typically set to twice the hardware watchdog's actual timeout duration. This conservative calculation ensures that the surviving, healthy cluster partition waits long enough for the isolated node's watchdog to definitively expire and reboot the machine before Pacemaker proceeds with the dangerous task of promoting a standby database to primary⁴⁷.

pg_auto_failover: Rethinking Fencing and the CAP Theorem

Maintained extensively by the engineering team at Citus Data (now part of Microsoft), the `pg_auto_failover` extension takes a distinctly different architectural approach, prioritizing operational simplicity through a unique architecture orchestrated by an independent, PostgreSQL-based "Monitor" node⁵².

Rather than relying on a clustered DCS like etcd, or complex Corosync network rings, the lightweight `pg_autoctl` keeper agents running on the PostgreSQL primary and standby nodes report their current state directly to the Monitor via standard PostgreSQL protocol SQL queries. This allows the system to utilize standard database concurrency, transaction support, and durability for state management⁵².

Regarding split-brain prevention and fencing, `pg_auto_failover` utilizes software-level self-fencing. If a network partition occurs and the primary node cannot reach the Monitor node or the secondary standby node within the defined `NETWORK_PARTITION_TIMEOUT`, it interprets this isolation as a definitive split-brain scenario. The primary's `pg_autoctl` process aggressively terminates the local PostgreSQL instance and transitions to a `DEMOTE_TIMEOUT` state⁵⁵.

Crucially, however, `pg_auto_failover` does not integrate natively with the `/dev/watchdog` hardware interface to execute ungraceful physical reboots. It fundamentally relies on the operating system's ability to cleanly stop the postgres process via software signals⁵². If the primary server is experiencing a total OS freeze or heavy I/O deadlock, this software shutdown may fail, leaving the system vulnerable. As explicitly noted by the project maintainers, the CAP theorem dictates unavoidable compromises between Consistency and Availability. The maintainers of `pg_auto_failover` deliberately opt to avoid the operational complexity of automatic hardware fencing, reasoning that fencing isolated nodes automatically can lead to unnecessary downtime. Instead, they suggest that enterprise users requiring absolute, mathematically guaranteed consistency must deploy continuous backup, synchronous replication, and strict shared-nothing anti-affinity configurations, relying on human intervention or manual fencing for the most complex edge cases⁵⁶.

repmgr: Administrator-Defined STONITH

`repmgr` (Replication Manager) by EnterpriseDB is a mature, long-standing tool suite designed

to manage streaming replication. While repmgrd (the active daemon component) possesses the capability to automate failover, it does not mandate a specific consensus backend or prescribe a rigid fencing methodology out of the box⁵⁸.

Instead of native watchdog integration, repmgr delegates the immense responsibility of STONITH directly to the system administrator via highly customizable execution hooks, primarily the `promote_command` and `follow_command` shell scripts⁵⁸. To prevent split-brain effectively, administrators must engineer and embed their own custom fencing logic—whether invoking an IPMI tool command to an out-of-band management interface, triggering an intelligent PDU to cut physical power, or interacting with a hypervisor via a `virsh destroy` command—directly into the shell scripts executed prior to promotion³⁰.

While recent iterations of repmgr have introduced advanced configurations such as `primary_visibility_consensus=true` and the capability to deploy a witness server to help determine network isolations algorithmically, the actual physical isolation of the rogue primary relies entirely on the quality of the administrator's custom scripting and the resilience of the external data center infrastructure³⁰.

HA Solution	Consensus Mechanism	Primary Fencing Strategy	Hardware Watchdog Integration
Patroni	Distributed Configuration Store (etcd, Consul, ZooKeeper)	Lease expiration & Self-demotion	Deep native integration via <code>linux.py</code> and <code>/dev/watchdog</code> .
Pacemaker + SBD	Corosync Quorum	STONITH (Shoot The Other Node In The Head)	Native via SBD daemon; essential for Diskless SBD deployments.
pg_auto_failover	Monitor Node (Postgres-based protocol)	Process termination (Software Self-fencing)	None natively; relies entirely on OS-level process management.
repmgr	Internal tables & Optional Witness server	Admin-defined script execution	Delegated to custom shell scripts invoked via <code>promote_command</code> .

Part 5: The Routing Layer - HAProxy and Keepalived

While watchdogs and distributed orchestrators manage the backend database state and ensure data integrity, the application routing layer must react instantaneously to these topology shifts. A standard, robust deployment utilizes HAProxy for Layer 4 TCP load balancing paired seamlessly with Keepalived for high-availability IP mobility³⁴.

HAProxy continuously polls the REST API exposed by the Patroni daemons on each node (typically checking the `/primary` and `/replica` endpoints over port 8008). It dynamically rewires routing logic, directing all critical write traffic strictly to the single node returning an HTTP 200 OK response on the `/primary` endpoint³⁴. Keepalived utilizes the Virtual Router Redundancy Protocol (VRRP) to float a single Virtual IP (VIP) between the HAProxy nodes. If an active proxy node fails, Keepalived instantly shifts the VIP to the passive proxy, ensuring uninterrupted application connectivity to the database cluster³⁴. Alternatively, tools like Pgpool-II provide similar functionality with added benefits like connection pooling and intelligent read/write query splitting, adapting to failovers orchestrated by Patroni⁶⁶.

Crucially, the entire routing layer relies implicitly on the integrity and success of the data layer's fencing mechanisms. If a watchdog fails to properly fence an isolated primary, and that rogue primary somehow remains reachable on the network due to an incomplete partition, a misconfigured HAProxy might inadvertently route writes to the degraded node, inducing split-brain logic directly at the application tier. Thus, the hardware watchdog serves as the ultimate, non-negotiable backstop, ensuring degraded nodes disappear entirely from both the data replication plane and the application routing plane by physically powering them down³.

Conclusion

The architecture of a high-availability PostgreSQL cluster is fundamentally incomplete without a definitive, uncompromising mechanism to resolve split-brain scenarios resulting from network partitions and system deadlocks. While advanced software-level orchestrators like Patroni, Pacemaker, and `pg_auto_failover` possess highly sophisticated logic to manage standard, graceful failovers, they remain inherently constrained by the underlying health, responsiveness, and scheduling capabilities of the operating system kernel and local hardware.

The strict integration of Linux watchdog mechanisms—whether through enterprise hardware BMCs, Intel TCO timers, or the virtualized `softdog` kernel module—provides the ultimate system fail-safe. By utilizing precise temporal mathematics, and ensuring the watchdog strictly triggers a hard reset prior to the expiration of distributed consensus leases, database administrators can mathematically guarantee that an isolated or frozen primary is definitively fenced. The meticulous implementation of these mechanisms transitions a database cluster from being merely highly available to being provably resilient against catastrophic data corruption.

Works cited

1. The Linux Kernel/Softdog Driver - Wikibooks, open books for an open world,

- https://en.wikibooks.org/wiki/The_Linux_Kernel/Softdog_Driver
2. Watchdog — VyOS 1.5.x (circinus) LTS,
<https://docs.vyos.io/en/1.5/configuration/system/watchdog.html>
 3. Patroni - Percona Distribution for PostgreSQL,
<https://docs.percona.com/postgresql/14/solutions/patroni-info.html>
 4. This simple Linux tweak fixes crashes automatically - and it costs me nothing - ZDNET,
<https://www.zdnet.com/article/how-linux-watchdog-reboots-my-frozen-system/>
 5. How components work together - Percona Distribution for PostgreSQL,
<https://docs.percona.com/postgresql/15/solutions/ha-components.html>
 6. Software-Emulated Watchdog Known Limitations - Red Hat Customer Portal,
<https://access.redhat.com/articles/7034141>
 7. Fencing for Fun and Profit with SBD - That Cluster Guy - Andrew Beekhof,
<https://www.beekhof.net/blog/2015/sbd-fun-and-profit>
 8. Configuring Disk-Based SBD in an Existing High Availability Cluster - SUSE Documentation,
<https://documentation.suse.com/sle-ha/16.0/html/HA-sbd-configuring-diskbased/index.html>
 9. Linux Watchdog Documentation,
<https://blog.fooool.net/wp-content/uploads/linuxdocs/watchdog.pdf>
 10. How to Configure Software Watchdog in Talos Linux - OneUptime,
<https://oneuptime.com/blog/post/2026-03-03-configure-software-watchdog-in-talos-linux/view>
 11. Configuring Diskless SBD in an Existing High Availability Cluster - SUSE Documentation,
https://documentation.suse.com/sle-ha/16.0/pdf/HA-sbd-configuring-diskless_en.pdf
 12. embedded linux - How to determine whether a hardware or a software watchdog is implemented on the machine? - Stack Overflow,
<https://stackoverflow.com/questions/24049385/how-to-determine-whether-a-hardware-or-a-software-watchdog-is-implemented-on-the>
 13. GCP VM spontaneous reboots (#7872) · Issue · gitlab-com/gitlab-com/gl-infra/production-engineering,
https://gitlab.com/gitlab-com/gl-infra/production-engineering/-/work_items/7872
 14. The Linux Watchdog driver API - Arch Linux カーネルドキュメント - Kusakata's Lair,
<http://doc.kusakata.com/watchdog/watchdog-api.html>
 15. Converting old watchdog drivers to the watchdog framework - The Linux Kernel documentation,
https://docs.kernel.org/watchdog/convert_drivers_to_kernel_api.html
 16. Watchdog - SigmaStarDocs,
<https://wx.comake.online/doc/d8clf27cnes2-SSD20X/customer/development/reference/watchdog.html>
 17. Can I send a heart beat to hardware watch dog from my own program?,
<https://raspberrypi.stackexchange.com/questions/1517/can-i-send-a-heart-beat-to-hardware-watch-dog-from-my-own-program>

18. watchdog - inappropriate ioctl for device on RHEL 8 ppc64le. · Issue #2265 - GitHub, <https://github.com/patroni/patroni/issues/2265>
19. systemd-system.conf(5) - Debian Manpages, <https://manpages.debian.org/testing/systemd/systemd-system.conf.5.en.html>
20. Leveraging Systemd for Hardware Watchdog Control in Embedded Linux, <https://cornersoftsolutions.com/leveraging-systemd-for-hardware-watchdog-control-in-embedded-linux/>
21. systemd for Administrators, Part XV - Opointer.net, <http://Opointer.de/blog/projects/watchdog.html>
22. en:general_information:hw-watchdog-systemd - TQ Support Wiki, https://support.tq-group.com/en/general_information/hw-watchdog-systemd
23. Using watchdog and sd-notify functionality for systemd in Python 3 - blog of stigok, <https://blog.stigok.com/2020/01/26/sd-notify-systemd-watchdog-python-3.html>
24. systemd watchdog for frozen application [closed] - Stack Overflow, <https://stackoverflow.com/questions/73840404/systemd-watchdog-for-frozen-application>
25. High Availability PostgreSQL: Clustering with Patroni - DEV Community, <https://dev.to/prezaei/high-availability-postgresql-clustering-with-patroni-5043>
26. Watchdog support — Patroni 4.1.3 documentation, <https://patroni.readthedocs.io/en/latest/watchdog.html>
27. Configuring Diskless SBD in an Existing High Availability Cluster - SUSE Documentation, <https://documentation.suse.com/sle-ha/16.0/html/HA-sbd-configuring-diskless/>
28. PostgreSQL HA with Patroni and HAProxy | JusDB Blog, <https://www.jusdb.com/blog/postgresql-ha-patroni-haproxy>
29. HA multi datacenter — Patroni 4.1.4 documentation, https://patroni.readthedocs.io/en/latest/ha_multi_dc.html
30. PG Phriday: Isolating Postgres with repmgr - EDB, <https://www.enterprisedb.com/blog/pg-phriday-isolating-postgres-repmgr>
31. PDF - Patroni Documentation, https://patroni.readthedocs.io/_/downloads/en/latest/pdf/
32. Patroni and etcd - AWS Prescriptive Guidance, <https://docs.aws.amazon.com/prescriptive-guidance/latest/migration-databases-postgresql-ec2/ha-patroni-etcd-considerations.html>
33. OpenBSD Ports Readme: port databases/patroni, <https://openports.pl/path/databases/patroni>
34. Set Up PostgreSQL High Availability with Patroni and HAProxy - ComputingForGeeks, <https://computingforgeeks.com/postgresql-high-availability-patroni-etcd-haproxy/>
35. Release notes — Patroni 4.1.3 documentation, <https://patroni.readthedocs.io/en/latest/releases.html>
36. YAML Configuration Settings — Patroni 3.0.1 documentation - Read the Docs, <https://enterprisedbpatroni.readthedocs.io/en/stable/SETTINGS.html>

37. patroni.watchdog.linux module,
<https://access.crunchydata.com/documentation/patroni/latest/modules/patroni.watchdog.linux/>
38. patroni/patroni/watchdog/linux.py at master - GitHub,
<https://github.com/zalando/patroni/blob/master/patroni/watchdog/linux.py>
39. Every few months, the Patroni API becomes unresponsive. · Issue #3474 - GitHub,
<https://github.com/patroni/patroni/issues/3474>
40. PostgreSQL HA with Patroni: Your Turn to Test Failure Scenarios - Percona,
<https://www.percona.com/blog/postgresql-ha-with-patroni-your-turn-to-test-failure-scenarios/>
41. postgres watchdog - PGConf,
[https://pgconf.ru/rmedia/2024/04/18/15/985/pg-conf-2024-watchdogs%20\(1\).pdf](https://pgconf.ru/rmedia/2024/04/18/15/985/pg-conf-2024-watchdogs%20(1).pdf)
42. usage of consul.checks for patroni (#8761) · Issue - GitLab,
<https://gitlab.com/gitlab-com/gl-infra/infrastructure/-/issues/8761>
43. Patroni Failure Stories or How to crash your PostgreSQL cluster. Alexey Lesovsky,
<https://prohoster.info/en/blog/administrirovanie/patroni-failure-stories-or-how-to-crash-your-postgresql-cluster-aleksej-lesovskij>
44. Corosync – Talip Hakan Ozturk's ORACLE BLOG - WordPress.com,
<https://taliphakanozturken.wordpress.com/tag/corosync/>
45. PostgreSQL High Availability: Automatic Failover - ScaleGrid,
<https://scalegrid.io/blog/managing-high-availability-in-postgresql-part-1/>
46. PostgreSQL Automatic Failover - Cluster administration under Debian 8 - GitHub Pages,
<https://clusterlabs.github.io/PAF/Debian-8-admin-cookbook.html>
47. Storage protection and SBD | Administration Guide | SLE HA 15 SP6 - SUSE Documentation,
<https://documentation.suse.com/de-de/sle-ha/15-SP6/html/SLE-HA-all/cha-ha-storage-protect.html>
48. Using SBD With Pacemaker - ClusterLabs,
https://projects.clusterlabs.org/w/fencing/using_sbd_with_pacemaker/
49. Configure diskless DRBD quorum - LINBIT Community,
<https://forums.linbit.com/t/configure-diskless-drbd-quorum/1162>
50. [ClusterLabs] SBD as watchdog daemon,
<https://lists.clusterlabs.org/pipermail/users/2019-April/025630.html>
51. Error: Unexpected state for instance "pgsqld" (returned 9) · Issue #173 · ClusterLabs/PAF,
<https://github.com/ClusterLabs/PAF/issues/173>
52. pg_auto_failover/docs/operations.rst at main - GitHub,
https://github.com/hapostgres/pg_auto_failover/blob/main/docs/operations.rst
53. Fixing Standby Slot Problems with pg_auto_failover - OpenSource DB,
https://opensource-db.com/dealing-with-standby-slot-problems-in-pg_auto_failover/
54. pg_replica 0.5.0: Consensus-driven automatic failover for PostgreSQL (embedded Raft control plane) / PostgreSQL Extension Network - PGXN,
https://pgxn.org/dist/pg_replica/0.5.0/
55. Design decisions regarding to fault-tolerance · Issue #12 · hapostgres/pg_auto_failover,

- https://github.com/hapostgres/pg_auto_failover/issues/12
56. Split-brain in case of network partition #7462 - GitHub,
<https://github.com/cloudnative-pg/cloudnative-pg/discussions/7462>
 57. [Bug]: Split-brain in case of network partition · Issue #7407 - GitHub,
<https://github.com/cloudnative-pg/cloudnative-pg/issues/7407>
 58. 13.1. repmgrd basic configuration,
<https://www.repmgr.org/docs/4.2/repmgrd-basic-configuration.html>
 59. repmgr 5.5.0 Documentation, <https://www.repmgr.org/docs/current/repmgr.html>
 60. 13.1. repmgrd configuration,
<https://www.repmgr.org/docs/5.0/repmgrd-basic-configuration.html>
 61. PostgreSQL Automatic Failover - Fencing,
<https://clusterlabs.github.io/PAF/fencing.html>
 62. How to Set Up a Highly Available PostgreSQL 17 Cluster using Patroni -
postgreshelp -, <https://postgreshelp.com/postgresql-17-ha-with-patroni/>
 63. Building a Highly Available PostgreSQL Cluster with Patroni, etcd, and PgBouncer
- Medium,
<https://medium.com/@yahya.muhaned/building-a-highly-available-postgresql-cluster-with-patroni-etcd-and-pgbouncer-f8e363342bf4>
 64. How to Setup PostgreSQL High Availability using Patroni & Etcd & HAProxy &
Keepalived for Debian (Ubuntu)? | SERHAT CELIK DATABASE BLOG,
<https://serhatcelik.wordpress.com/2025/05/30/how-to-setup-postgresql-high-availability-using-patroni-etcd-haproxy-keepalived-for-debian-ubuntu/>
 65. ProxySQL for PostgreSQL — the failover model,
<https://proxysql.com/blog/proxysql-postgresql-failover-primer/>
 66. Demystifying High Availability: PostgreSQL with Patroni and Pgpool-II on Ubuntu -
Medium,
<https://medium.com/@joaovic32/demystifying-high-availability-postgresql-with-patroni-and-pgpool-ii-on-ubuntu-428c91a55b1a>